

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ

«Национальный исследовательский университет ИТМО»

Факультет безопасности информационных технологий

Дисциплина:

«Программирование»

ОТЧЁТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №4

Работа с файлами и структуры данных на основе указателей

Выполнил:

Студент группы N3150

Пахомов Владимир Юрьевич



Проверил:

Грозов В. А.

Санкт-Петербург

2025г.

Оглавление

Задание. Вариант 1-9-10-2.....	3
Make-файл.....	4
Примеры работы программы на различных исходных данных.....	5
Исходный текст программы с комментариями.....	6

Задание. Вариант 1-9-10-2

Таблица 1. Тип списка:

Односвязный список

Таблица 2. Формат данных в строке:

ISBN-13

С проверкой по алгоритму ISBN-13. Примеры:

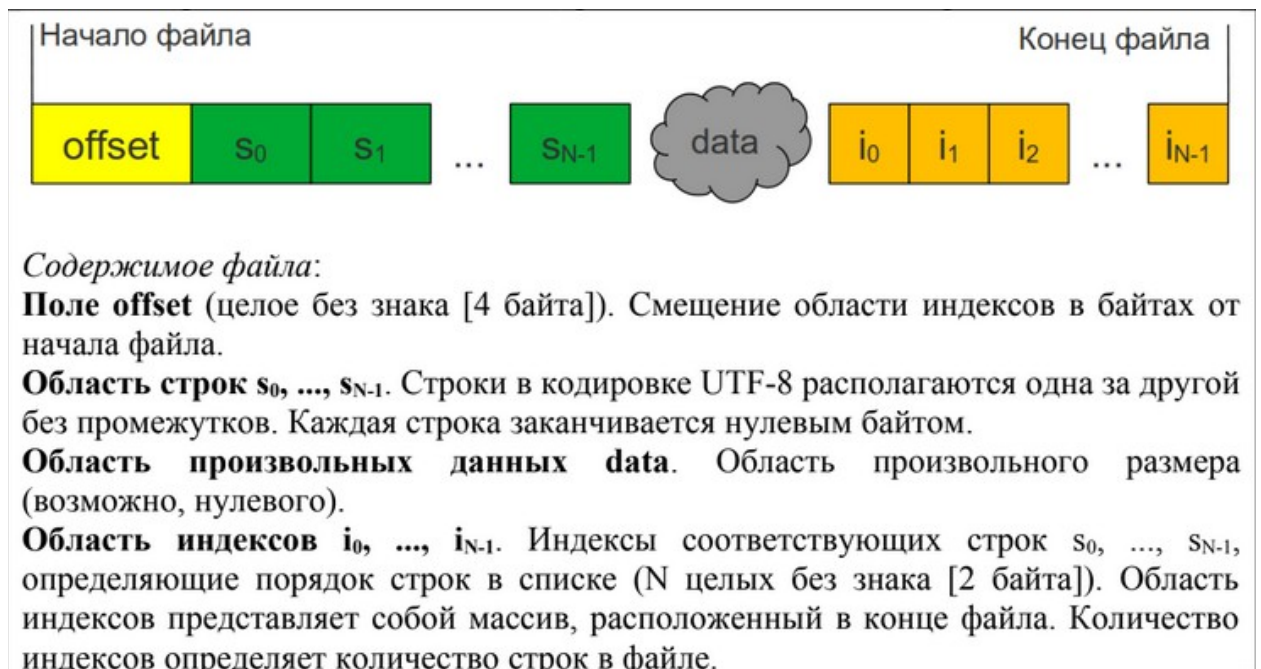
978-0-306-40615-7, 978-2-266-11156-0

Таблица 3. Команда:

Команда: min_first

Описание: Переместить элемент с максимальным значением (или все элементы с максимальным значением, если их в списке несколько) в начало списка, не изменяя порядок других элементов.

Таблица 4. Формат файла:



Make-файл

```
.PHONY: all clean
```

```
APP=prg4vypN3150
```

```
CFLAGS=-Wall -Wextra -Werror -g
```

```
all: $(APP)
```

```
$(APP): $(APP).c
```

```
gcc -o $(APP) $(CFLAGS) $(APP).c
```

```
clean:
```

```
rm $(APP)
```

Примеры работы программы на различных исходных данных

Python файл:

```
❌ ~ /C_Workspace/prg4vypN3150 python prg4vypN3150_gen.py
Ошибка: выходной файл не задан.
❌ ~ /C_Workspace/prg4vypN3150 python prg4vypN3150_gen.py in.txt
Файл 'in.txt' успешно сгенерирован.
❌ ~ /C_Workspace/prg4vypN3150 python prg4vypN3150_gen.py -n=21.1 in.txt
usage: prg4vypN3150_gen.py [-h] [-v] [-n N] [filename]
prg4vypN3150_gen.py: error: argument -n: invalid int value: '21.1'
❌ ~ /C_Workspace/prg4vypN3150 python prg4vypN3150_gen.py -n=-123 in.txt
Ошибка: количество строк должно быть положительным.
❌ ~ /C_Workspace/prg4vypN3150 python prg4vypN3150_gen.py -n=10 in.txt
Файл 'in.txt' успешно сгенерирован.
❌ ~ /C_Workspace/prg4vypN3150 python prg4vypN3150_gen.py -v
Пахомов Владимир Юрьевич, гр. N3150
Вариант: 1-9-10-2
❌ ~ /C_Workspace/prg4vypN3150 python prg4vypN3150_gen.py -v in.txt
Пахомов Владимир Юрьевич, гр. N3150
Вариант: 1-9-10-2
❌ ~ /C_Workspace/prg4vypN3150
```

С файл:

```
❌ ~ /C_Workspace/prg4vypN3150 ./prg4vypN3150
Ошибка: не задано имя файла.
❌ ~ /C_Workspace/prg4vypN3150 ./prg4vypN3150 in.txt
pop_back
push_front 978-0-306-40615-7
push_back 128938213
Ошибка: неправильный ISBN-13 формат: 128938213
push_front 978-0-306-40123-0
Ошибка: неправильный ISBN-13 формат: 978-0-306-40123-0
min_first
delete
Ошибка: неподдерживаемая команда: 'delete'.
dump
0x00005caed943d610 0x00005caed943d490 978-0-306-40615-7
0x00005caed943d490 0x00005caed943d4c0 979-0-696-47503-9
0x00005caed943d4c0 0x00005caed943d4f0 979-4-721-14333-7
0x00005caed943d4f0 0x00005caed943d520 979-4-008-60621-0
0x00005caed943d520 0x00005caed943d550 979-3-627-94827-5
0x00005caed943d550 0x00005caed943d580 978-1-094-43833-7
0x00005caed943d580 0x00005caed943d5b0 978-0-380-46385-5
0x00005caed943d5b0 0x00005caed943d5e0 978-0-833-23182-6
0x00005caed943d5e0 0x0000000000000000 979-1-348-46280-3
dump out.txt
exit
❌ ~ /C_Workspace/prg4vypN3150
```

Исходный текст программы с комментариями

Python файл:

```
#!/usr/bin/python3
from sys import exit
from random import randint
from argparse import ArgumentParser
from struct import pack

def generate_isbn13():
    prefix = str(randint(978, 979))
    group = str(randint(0, 5))
    publisher = str(randint(0, 999)).zfill(3)
    title = str(randint(0, 99999)).zfill(5)

    digits = prefix + group + publisher + title
    total = sum(int(digits[i]) if i % 2 == 0 else int(digits[i]) * 3 for i in
range(len(digits)))
    checksum = (10 - (total % 10)) % 10
    return f"{prefix}-{group}-{publisher}-{title}-{checksum}"

def main():
    parser = ArgumentParser()
    parser.add_argument("-v", action="store_true")
    parser.add_argument("-n", type=int)
    parser.add_argument("filename", nargs="?")
    args = parser.parse_args()

    author = "Пахомов Владимир Юрьевич, гр. N3150\nВариант: 1-9-10-2";

    if args.v:
        print(author)
        exit(0)

    if not args.filename:
        print("Ошибка: выходной файл не задан.")
        exit(1)

    num_strings = args.n if args.n is not None else randint(10, 1000)
    if num_strings <= 0:
        print("Ошибка: количество строк должно быть положительным.")
        exit(2)

    strings = [generate_isbn13().encode("utf-8") + b"\x00" for _ in
range(num_strings)]

    data_size = randint(0, 1024)
    data = bytes(randint(0, 255) for _ in range(data_size))

    offset = 4
    string_positions = []
    current_pos = offset
    for s in strings:
        string_positions.append(current_pos)
        current_pos += len(s)

    data_pos = current_pos
```

```

current_pos += len(data)

index_positions = []
indices = []
for i in range(num_strings):
    index_positions.append(current_pos)
    indices.append(i)
    current_pos += 2

# Запись в файл
try:
    with open(args.filename, "wb") as f:
        f.write(pack("<I", offset))
        for s in strings:
            f.write(s)
        f.write(data)
        for idx in indices:
            f.write(pack("<H", idx))

    print(f"Файл'{args.filename}' успешно сгенерирован.")
    exit(0)

except Exception as e:
    print(f"Ошибка записи в файл: {e}")
    exit(3)

if __name__ == "__main__":
    main()

```

```

С файл:
#include <stdlib.h>
#include <stdio.h>
#include <string.h>

#include <ctype.h>
#include <stdint.h>

#define MAX_LINE 1024
#define MAX_ISBN_LEN 18 // ISBN-13, например "978-0-5791-81415-1" + \0

int issaved = 0;

typedef struct Node {
    char isbn[MAX_ISBN_LEN];
    struct Node* next;
    int index;
} Node;

// Список
typedef struct {
    Node* head;
    int count;
    int modified;
} List;

// Инициализация списка
void init_list(List* list) {
    list->head = NULL;
    list->count = 0;
    list->modified = 0;
}

// Освобождение списка
void free_list(List* list) {
    Node* current = list->head;
    while (current) {
        Node* temp = current;
        current = current->next;
        free(temp);
    }
    list->head = NULL;
    list->count = 0;
    list->modified = 0;
}

// Проверка на ISBN-13
int validate_isbn13(const char* isbn) {
    if (strlen(isbn) != MAX_ISBN_LEN - 1) return 0;
    if (isbn[3] != '-' || isbn[5] != '-' || isbn[9] != '-' || isbn[15] != '-') return
0;

    int num_digits = 13;
    char digits[num_digits];
    int k = 0;
    for (int i = 0; i < MAX_ISBN_LEN - 1; i++) {

```



```

        if (i == 3 || i == 5 || i == 9 || i == 15) continue;
        if (!isdigit(isbn[i])) {
            return 0;
        }
        digits[k++] = isbn[i] - '0';
    }

    int sum = 0;
    for (int i = 0; i < num_digits-1; i++) {
        sum += (i % 2 == 0) ? digits[i] : digits[i] * 3;
    }
    int checksum = (10 - (sum % 10)) % 10;
    return checksum == digits[num_digits-1];
}

// Создание новой строки
Node* create_node(const char* isbn) {
    Node* node = (Node*)malloc(sizeof(Node));
    if (!node) {
        fprintf(stderr, "Ошибка: не удалось выделить память.\n");
        exit(3);
    }
    strncpy(node->isbn, isbn, MAX_ISBN_LEN);
    node->isbn[MAX_ISBN_LEN - 1] = '\0';
    node->next = NULL;
    node->index = 0;
    return node;
}

// Добавить строку в начало
void push_front(List* list, const char* isbn) {
    if (!validate_isbn13(isbn)) {
        fprintf(stderr, "Ошибка: неправильный ISBN-13 формат: %s\n", isbn);
        return;
    }
    Node* node = create_node(isbn);
    node->next = list->head;
    list->head = node;
    list->count++;
    list->modified = 1;
}

// Добавить строку в конец
void push_back(List* list, const char* isbn) {
    if (!validate_isbn13(isbn)) {
        fprintf(stderr, "Ошибка: неправильный ISBN-13 формат: %s\n", isbn);
        return;
    }
    Node* node = create_node(isbn);
    if (!list->head) {
        list->head = node;
    } else {
        Node* current = list->head;
        while (current->next) {
            current = current->next;
        }
    }
}

```

```

        current->next = node;
    }
    list->count++;
    list->modified = 1;
}

// Удалить строку из начала
void pop_front(List* list) {
    if (!list->head) return;
    Node* temp = list->head;
    list->head = list->head->next;
    free(temp);
    list->count--;
    list->modified = 1;
}

// Удалить строку с конца
void pop_back(List* list) {
    if (!list->head) return;
    if (!list->head->next) {
        free(list->head);
        list->head = NULL;
        list->count--;
        list->modified = 1;
        return;
    }
    Node* current = list->head;
    while (current->next->next) {
        current = current->next;
    }
    free(current->next);
    current->next = NULL;
    list->count--;
    list->modified = 1;
}

// Записать список в поток вывода
void dump_list(List* list, const char* filename) {
    FILE* fp = stdout;
    if (filename) {
        fp = fopen(filename, "w");
        if (!fp) {
            fprintf(stderr, "Ошибка: Не удалось открыть файл %s.\n", filename);
            return;
        }
        issaved = 1;
    }

    Node* current = list->head;
    int i = 0;
    while (current) {
        uintptr_t addr = (uintptr_t)(current);
        uintptr_t next_addr = current->next ? (uintptr_t)(current->next) : 0;
        fprintf(fp, "0x%016lx 0x%016lx %s\n", addr, next_addr, current->isbn);
        current = current->next;
        i++;
    }
}

```

```

    }

    if (filename) fclose(fp);
}

// Переместить минимальные элементы в начало списка
void min_first(List* list) {
    if (!list->head || !list->head->next) return;

    // Нахождение минимального ISBN-13
    Node* current = list->head;
    char* min_isbn = current->isbn;
    while (current) {
        if (strcmp(current->isbn, min_isbn) < 0) {
            min_isbn = current->isbn;
        }
        current = current->next;
    }

    Node* min_nodes = NULL;
    Node* other_nodes = NULL;
    Node* min_tail = NULL;
    Node* other_tail = NULL;

    current = list->head;
    while (current) {
        Node* next = current->next;
        current->next = NULL;
        if (strcmp(current->isbn, min_isbn) == 0) {
            if (!min_nodes) {
                min_nodes = current;
                min_tail = current;
            } else {
                min_tail->next = current;
                min_tail = current;
            }
        } else {
            if (!other_nodes) {
                other_nodes = current;
                other_tail = current;
            } else {
                other_tail->next = current;
                other_tail = current;
            }
        }
        current = next;
    }

    if (min_tail) {
        min_tail->next = other_nodes;
        list->head = min_nodes;
    } else {
        list->head = other_nodes;
    }
    list->modified = 1;
}

```

```

// Чтение файла
int read_file(List* list, const char* filename, uint8_t** data_section, size_t*
data_size) {
    FILE* fp = fopen(filename, "rb");
    if (!fp) return 0;

    fseek(fp, 0, SEEK_END);
    long size = ftell(fp);
    if (size == 0) {
        fclose(fp);
        return 0;
    }
    fseek(fp, 0, SEEK_SET);

    // Чтение offset
    uint32_t offset;
    if (fread(&offset, sizeof(uint32_t), 1, fp) != 1) {
        fclose(fp);
        fprintf(stderr, "Ошибка: Неправильный формат файла.\n");
        return 1;
    }

    // Чтение строк
    fseek(fp, offset, SEEK_SET);
    long pos = offset;
    char buffer[MAX_ISBN_LEN];
    int string_count = 0;
    while (1) {
        int i = 0;
        char c;
        while (fread(&c, 1, 1, fp) && c != '\0' && i < MAX_ISBN_LEN) {
            buffer[i++] = c;
        }
        if (c != '\0') break; // Конец строк
        buffer[i] = '\0';
        pos = ftell(fp);
        if (!validate_isbn13(buffer)) {
            fclose(fp);
            fprintf(stderr, "Ошибка: неправильный формат ISBN-13: %s\n", buffer);
            return 1;
        }
        push_back(list, buffer);
        string_count++;
    }

    // Позиция начала данных
    long data_start = pos;
    fseek(fp, -string_count * sizeof(uint16_t), SEEK_END);
    long indices_start = ftell(fp);
    if (indices_start <= data_start) {
        fclose(fp);
        fprintf(stderr, "Ошибка: неправильный формат файла.\n");
        return 1;
    }
}

```

```

    }

    // Выделяем память и читаем секцию данных
    *data_size = indices_start - data_start;
    *data_section = malloc(*data_size);
    if (!*data_section) {
        fclose(fp);
        fprintf(stderr, "Ошибка: не удалось выделить память.\n");
        return 1;
    }

    fseek(fp, data_start, SEEK_SET);
    if (fread(*data_section, 1, *data_size, fp) != *data_size) {
        fclose(fp);
        free(*data_section);
        *data_section = NULL;
        *data_size = 0;
        fprintf(stderr, "Ошибка: не удалось прочитать секцию данных.\n");
        return 1;
    }

    // Чтение индексов
    Node* current = list->head;
    for (int i = 0; i < string_count && current; i++) {
        uint16_t index;
        if (fread(&index, sizeof(uint16_t), 1, fp) != 1) {
            fclose(fp);
            free(*data_section);
            *data_section = NULL;
            *data_size = 0;
            fprintf(stderr, "Ошибка: неправильный формат файла.\n");
            return 1;
        }
        current->index = index;
        current = current->next;
    }

    fclose(fp);
    return 0;
}

// Write file
void write_file(List* list, const char* filename, uint8_t* data_section, size_t
data_size) {
    FILE* fp = fopen(filename, "wb");
    if (!fp) {
        fprintf(stderr, "Ошибка: не удалось открыть файл %s.\n", filename);
        exit(2);
    }

    if (list->count == 0) {
        fclose(fp);
        return;
    }

```

```

    }

    // Запись offset
    uint32_t offset = 4;
    fwrite(&offset, sizeof(uint32_t), 1, fp);

    // Запись строк
    Node* current = list->head;
    int i = 0;
    while (current) {
        current->index = i++;
        fwrite(current->isbn, strlen(current->isbn) + 1, 1, fp);
        current = current->next;
    }

    // Запись данных
    if (data_section && data_size > 0) {
        fwrite(data_section, 1, data_size, fp);
    }

    // Запись индексов
    current = list->head;
    while (current) {
        uint16_t index = current->index;
        fwrite(&index, sizeof(uint16_t), 1, fp);
        current = current->next;
    }

    fclose(fp);
}

// Обработка команд
void handle_commands(List* list, const char* filename, uint8_t* data_section, size_t
data_size) {
    char line[MAX_LINE];
    while (fgets(line, MAX_LINE, stdin)) {
        line[strcspn(line, "\n")] = '\0';
        if (strlen(line) == 0) continue;

        char* token = strtok(line, " ");
        if (!token) continue;

        if (strcmp(token, "push_front") == 0) {
            token = strtok(NULL, " ");
            while (token) {
                push_front(list, token);
                token = strtok(NULL, " ");
            }
        }
        else if (strcmp(token, "push_back") == 0) {
            token = strtok(NULL, " ");
            while (token) {
                push_back(list, token);
                token = strtok(NULL, " ");
            }
        }
    }
}

```

```

        else if (strcmp(token, "pop_front") == 0) {
            pop_front(list);
        }
        else if (strcmp(token, "pop_back") == 0) {
            pop_back(list);
        }
        else if (strcmp(token, "dump") == 0) {
            token = strtok(NULL, " ");
            dump_list(list, token);
        }
        else if (strcmp(token, "min_first") == 0) {
            min_first(list);
        }
        else if (strcmp(token, "exit") == 0) {
            break;
        }
        else {
            fprintf(stderr, "Ошибка: неподдерживаемая команда: '%s'.\n", token);
        }
    }

    if (list->modified && issaved == 0) {
        write_file(list, filename, data_section, data_size);
    }
}

```

```

int main(int argc, char* argv[]) {
    // Проверка запуска с переменной среды, включающей отладочный вывод.
    // Пример запуска с установкой переменной LAB4DEBUG в 1:
    // $ LAB4DEBUG=1 ./prg4abcNXXXXXX test.txt
    char *DEBUG = getenv("LAB4DEBUG");
    if (DEBUG) {
        fprintf(stderr, "Включен вывод отладочных сообщений\n");
    }

    char *author = "Пахомов Владимир Юрьевич, гр. N3150\nВариант: 1-9-10-2";

    if (argc == 2) {
        if (strcmp(argv[1], "-v") == 0) {
            fprintf(stdout, "%s\n", author);
            exit(0);
        }
    }
    if (argc == 1) {
        fprintf(stderr, "Ошибка: не задано имя файла.\n");
        exit(1);
    }
    if (argc > 2) {
        fprintf(stderr, "Использование: %s [-v] имя_файла.\n", argv[0]);
        exit(2);
    }

    List list;
    init_list(&list);
}

```

```

uint8_t* data_section = NULL;
size_t data_size = 0;

if (read_file(&list, argv[1], &data_section, &data_size) != 0) {
    free_list(&list);
    exit(1);
}

handle_commands(&list, argv[1], data_section, data_size);

free_list(&list);
free(data_section);
return EXIT_SUCCESS;
}

// Вариант 1-9-10-2
// 1 - Односвязный список
// 9 - ISBN-13 978-0-306-40615-7, 978-2-266-11156-0
// 10 - Команда: min_first
//      Описание: Переместить элемент с минимальным значением (или все элементы с
//      минимальным значением, если их в списке несколько) в начало списка, не
//      изменяя порядок других элементов.
// 2 - offset(4 байта) + строки + данные + индексы(2 байта каждый)

```